

Hybrid Matching Algorithm for Personal Names

CIHAN VAROL, Sam Houston State University

COSKUN BAYRAK, University of Arkansas at Little Rock

Companies acquire personal information from phone, World Wide Web, or email in order to sell or send an advertisement about their product. However, when this information is acquired, moved, copied or edited, the data may lose its quality. Often, the use of data administrators or a tool that has limited capabilities to correct the mistyped information can cause many problems. Moreover, most of the correction techniques are particularly implemented for the words used in daily conversations. Since personal names have different characteristics compared to general text, a hybrid matching algorithm (PNRS) which employs phonetic encoding, string matching and statistical facts to provide a possible candidate for misspelled names is developed. At the end, the efficiency of the proposed algorithm is compared with other well known spelling correction techniques.

Categories and Subject Descriptors: H.1.1 [Models and Principles]: Systems and Information Theory—*Information theory; value of information*; I.2 [Artificial Intelligence]: Natural Language Processing - *Language parsing and understanding; Text Analysis*

General Terms: Algorithms, Experimentation, Performance

Additional Key Words and Phrases: Data Quality, Edit-Distance, Information Quality, Phonetic Strategy, Spelling Correction

ACM Reference Format:

Varol, C., and Bayrak, C. 2012. Hybrid Matching Algorithm for Personal names. ACM Journal of Data and Information Quality, Issue 3, Volume 4,

DOI= XXXXXXXX

1. INTRODUCTION

Organizations today depend on data to gain advantage on the decision making process, while higher the quality of the data on which their decisions are made the better the results are. On the other hand, decisions made from low quality data can cost an organization a lot of money. Therefore, researchers have been developing and testing new knowledge in the information quality field as well as develop information quality benchmarking standards for the last two decades [Wang et al. 1993; Madnick et al. 2009; Strong et al. 1997; Kahn et al. 2002].

The cost for poor quality data to the organizations varies. For example, The Data Warehousing Institute estimates that the cost of low quality customer data is about \$611 billion a year in postage for U.S. businesses [Eckerson 2002]. According to Gary McSherry, a Sogeti data management specialist, poor quality data in IT systems is costing Irish organizations over \$14 billion per year [Sogeti 2009]. Another example was provided by Frank Dravis where a pizza chain wants to send an offer through mail to the top 20 percent of its customers but they missed its target by \$0.5M because of bad customer data [Dravis 2002]. Data quality can also end up costing lives: in 1986, NASA space shuttle Challenger's solid rocket booster joint seals burst, leading to an explosion that killed seven people. NASA used a flawed decision-making process to approve the launch of the shuttle, which was caused by incomplete and misleading information [Rogers 1986]. All these examples can be caused by a whole host of factors, such as wrong numbers, addresses, Social Security numbers, date of births, where wrong personal names is being one of them.

Data collection methods vary from source to source by format, volume, and media type. Therefore, it is advantageous to deploy customized data hygiene techniques to standardize the data

Author's addresses: C. Varol, Department of Computer Science, Sam Houston State University; C. Bayrak, Computer Science Department, University of Arkansas at Little Rock

Permission to make digital or hardcopies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credits permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

@2010 ACM 1539-9087/2010/03-ART39 \$15.00

DOI10.1145/0000000.0000000 <http://doi.acm.org/10.1145/0000000.0000000>

for meaning fullness and usefulness based on the organization. Any set of data may be considered to have a level of accuracy which directly impacts the usefulness of that data. Names are also important pieces of information when databases are duplicated and when data sets are linked or integrated and no unique entity identifiers are available [Christen et al. 2004]. For instance, the US government committed over \$6 billion dollars to the hospitals for the care of undocumented immigrants in 2010 [Martin and Ruark 2010]. Since those individuals do not have Social Security Numbers, with only using their personal name they can continue to get treatment without alarming the hospital officials for the previous balances. Therefore, a correct name search is vital for accessing hospital records of individuals who do not have Social Security Numbers in hand. Another example can be given for the People Search services. Over the last few years People Search has emerged as an important service to the community. Unlike regular web searches related with variety of products, news, shopping, and information, People Search is a search conducted in order to obtain information about people [Udupa and Kumar 2010]. However, a good amount of search queries conducted on personal names is misspelled likewise a regular web search [Udupa and Kumar 2010]. Having a robust name spelling correction algorithm will reduce the time and effort needed by users to find people that they are searching for it. In other words, the more accurate a piece of information, the more one may depend on it [Varol and Bayrak 2005].

When collecting data from surveys, advertising campaigns or the like, knowledge of the accuracy of each individual piece of data as well as the aggregate accuracy of the whole, can help both in making use of the data, and determination of the effectiveness of various methods of data collection. Problems arise when a large amount of data is collected and each piece is subject to some kind of control mechanism. For a human being carrying out this task could take days, months or even years depending on the amount of data. Moreover, labor work is error-prone and may lead to different results for different investigators [Varol et al. 2005]. Automating the process as much as possible obviously minimizes the amount of time researchers must devote to the task.

The problem of devising algorithms and techniques for automatically correcting words in text has become a perennial research challenge since 1960s. However, often, the use of a tool that has limited capabilities to correct the mistyped information can cause many problems. Moreover, most of these techniques are particularly implemented for the words used in daily conversations. Since personal names have different characteristics compared to general text, a composite algorithm which employs phonetic encoding, string matching and statistical facts need to be developed based on the potential sources of variations and errors that occur in a name. For this reason, the *Personal Name Recognizing Strategy* (PNRS) is created to provide the closest match for misspelled names.

The next section of this paper is a detailed discussion of common personal name spelling errors. An overview of the spelling correction techniques is discussed in Section 3. In Section 4, the PNRS algorithm will be introduced. The comparison of the PNRS algorithm with other correction techniques with using a real-world data set is presented in Section 5.

2. TYPES AND SOURCES OF ERROR IN PERSONAL NAMES

The primary type of errors is an *isolated-word error* [Kukich 1992]. This is a single misspelled or mistyped word that can be captured with simple techniques. As the name suggests, isolated-word errors are invalid strings, properly identified and isolated as incorrect representations of a valid word [Becchetti and Ricotti 1999]. The primary isolated errors are as follows:

- Typographic errors
- Cognitive errors
- Phonetic errors

Typographic errors (also known as fat-fingering) occur when one letter is accidentally typed in place of another. For example, in the case of “Goerge” while trying to type “George”. These errors are based on the assumption that the writer or typist knows how to spell the word, but may have typed the word in a rush [Kukich 1992]. These errors do not affect the phonetic structure of a name but still poses a problem for matching. *Cognitive errors* refer to situations where the writer or typist chooses an incorrect spelling due to lack of knowledge of the correct one. As an example, the incorrect spelling of “Ralph” as “Rhalf” [Kukich 1992] can be used. *Phonetic errors* can be thought as a subset of cognitive errors. These errors are made when the writer substitutes letters into a word where the sound of it is mistakenly believed to be correct, which in fact leads to a

misspelling [Jurzik 2006]. As an example, the spelling of “*Gail*” as “*Gayle*” would be proper to use. *Optical Character Recognition (OCR) errors* are other type of errors that can arise from OCR misinterpretations of the original document [Taghya and Stovsky 2001]. These errors include the merging and splitting of words and characters, or incorrect framing of characters that usually results in one-to-many mappings, insertions of characters, deletions of characters, and rejections of characters due to low confidence levels in recognition.

Aside from OCR errors, *manual keyboard based data entry* can result in wrongly typed neighboring keys. In some cases the data administrators correct the mistake immediately. However, in often times these errors are not recognized, possibly due to limited time or by distractions of the person handling the data entry. Most of the cognitive and phonetic errors rise from *data entry over the telephone*. This is an additional problem in manual keyboard entry. The person doing the data entry may request the spelling of the name when he is on the phone, or assume a default spelling which is based on the data administrator’s knowledge and cultural background.

3. SPELLING CORRECTION TECHNIQUES

There are many isolated-word error correction applications, and these techniques entail the problem to three sub-problems: treat detection of an error, generation of candidate corrections, and ranking of candidate corrections as a separate process in sequence [Kukich 1992]. In most cases there is only one correct spelling for a particular word. However, there are often several valid possible name combinations for a particular one, such as ‘Aaron’ and ‘Erin’. Also using nicknames in daily life, for instance ‘Bob’ rather than ‘Robert’ make matching of personal names more challenging compared to general text.

Many variations for approximate string matching have been developed [Pfreifer et al. 1996; Zobel and Dart 1996; Gong and Chan 2006]. Although most of the techniques that are going to be discussed in this paper are particularly designed for general text, some of them are used as name spelling correction algorithm as well. Two main categories are defined for the techniques used for isolated-word error correction; pattern matching and phonetic matching.

3.1 Pattern Matching Techniques

Pattern matching techniques are commonly used as approximate string matching in dictionary based search [Hall and Dowling 1980; Jokinen et al. 1996; Navarro 2001], which is used for data linkage [Christen and Goiser 2006; Winkler 2006], duplicate detection [Cohen et al. 2003], information retrieval [Gong and Chan 2006] and correction of spelling errors [Kukich 1992]. Some of these, which are directly related to the presented work, are edit distance, rule-based techniques, n-gram based, Longest Common Substring, Jaro-Winkler algorithm, probabilistic techniques, and neural net.

3.1.1. Edit Distance Techniques. The most common pattern matching technique, *the edit distance* is defined as the smallest number of changes required for converting one string into another [Levenshtein 1965]. The edit distance from one string to another is calculated by the number of operations, such as replacements, insertions or deletions. Minimum edit distance techniques have been applied to virtually all spelling correction tasks, including text editing and natural language interfaces. The spelling correction accuracy varies with applications and algorithms. [Damerau 1990] reports a 95 percent correction rate for single-error misspellings for a test set of 964 misspellings of medium and long words (length 5 or more characters) while using a lexicon of 1,593 words. However, his overall correction rate was 84 percent when multi-error misspellings were counted. On the other hand, [Durhaiw et al. 1983] reports an overall 27 percent correction rate for a very simple, fast, and plain single-error correction algorithm accessing a keyword lexicon of about 100 entries. Although the rates seem low, the authors report a high degree of user satisfaction for this command language interface application due to their algorithm’s unobtrusiveness. All errors in the Damerau-Levenshtein metric are given the same cost (zero or one). However, some letters are more easily substituted with each other, due to e.g. keyboard layout, similar shapes, or phonetic similarity. For a more realistic distance measure, unusual errors may have a higher cost than common mistakes. As an example, both *Jasin* and *Jaon* have edit distance one to *Jason* according to the standard Damerau-Levenshtein metric, but the first string is

arguable the better match than the second. Based on this idea, statistical data from spelling errors is used to derive suitable distance costs between any two letters. In recent work, [Brill and Moore 2002] report experiments with modeling more powerful edit operations, allowing generic string-to-string edits. Moreover, additional heuristics are also used to complement techniques based on edit distance. For instance, in the case of typographic errors, the keyboard layout is very important. It is much more common to accidentally substitute a key by another if they are placed near each other on the keyboard. However, this approach is very sensitive, since the distribution of errors varies depending on way of input, the language used, and the kind of text involved.

3.1.2. Rule Based Techniques. *Rule-based techniques* attempt to use the knowledge gained from spelling error patterns and write heuristics that take advantage of this knowledge [Yannakoudakis and Fawthrop 1983]. SPEEDCOP [Raghayan et al. 1989], is an example of an error correction application using knowledge-based algorithms. SPEEDCOP is limited to single-error misspellings, motivated by research showing that over 80 percent of all spelling mistakes belong to this kind of error. The two knowledge based algorithms used by SPEEDCOP are generated for each entry in the dictionary. The dictionary is then sorted in key order. A misspelling is corrected by locating words with keys close to the keys of the misspelled word.

3.1.3. N-gram Based Techniques. The *character n-gram-based technique* coincides with the character n-gram analysis in non-word detection. However, instead of observing certain bi-grams and trigrams of letters that never or rarely occur, this technique can calculate the likelihood of one character following another and use this information to find possible correct word candidates [Ullman 1977].

3.1.4. Longest Common Substring. *Longest Common Sub-String algorithm* [Friedman and Sideli 1992] finds the longest common sub-string in the two strings being compared. For example, the two name strings ‘Martha’ and ‘Marhtas’ have a longest common sub-string ‘Marta’. The total length of the common sub-strings is 5 out of 6 and 7. A similarity measure can be calculated by dividing the total length of the common sub-strings by the minimum, maximum or average lengths of the two original strings. As shown with the example above, this algorithm is suitable for compound names that have words swapped. The time complexity of the algorithm, which is based on a dynamic programming approach is $O(|s1| \times |s2|)$ using $O(\min(|s1|, |s2|))$ space [Christen 2006].

3.1.5. Jaro-Winkler Distance. *Jaro* [Yancey 2005] is an algorithm commonly used in data linkage system. Recently this algorithm is being used to measure the distances between the words. The Jaro distance metric states that given two strings $s1$ and $s2$, their distance d_j is:

$$d_j = \frac{1}{3} \left(\frac{m}{|s1|} + \frac{m}{|s2|} + \frac{m-t}{m} \right)$$

where:

- m is the number of matching characters , and
- t is the number of transpositions

The *Winkler* [Yancey 2005] *algorithm* improves over the Jaro algorithm by applying ideas based on empirical studies which found that fewer errors typically occur at the beginning of names. The Winkler algorithm therefore increases the Jaro similarity measure for agreeing on initial characters (up to four).

3.1.6. Probabilistic Techniques and Neural Nets. Naturally, n-grams can be used to calculate probabilities and this has led to the *probabilistic techniques* demonstrated by [Lee 1999]. In particular, transition probabilities can be trained using n-grams from a large corpus and these n-grams can then represent the likelihood of one character following another. However, like rule-based systems the use of probabilistic information alone is not enough to achieve acceptable error correction rates [Kukich 1992]. *Neural net techniques* have emerged as likely candidates for spelling correctors due to their ability to do associative recall based on incomplete and noisy data. This means that they are trained on the spelling errors themselves and carry the ability to adapt to the specific spelling error patterns that they are trained upon [Trenkle and Vogt 1994]. The main

problem of neural nets is that running the learning cycles needed to obtain acceptable correction accuracy requires a very long time. Training time increases non-polynomial with dictionary size.

3.2 Phonetic Matching Techniques

All phonetic encoding techniques attempt to convert a name string into a code according to the way a name is pronounced. Therefore, this process is language dependent. Most of the designed techniques have been mainly developed based on English phonetic structure. However, several other techniques have been designed for other languages as well [Christen 2006]. Due to the relevance, here we only focus on the Soundex, Phonex, Phonix, and Metaphone algorithms.

3.2.1. Soundex. The *SOUNDEX*, which is used to correct phonetic spellings, maps a string into a key consisting of its first letter followed by a sequence of digits [Philips 1990]. All vowels, 'h', 'w' and 'y' are removed from the sequences to get the Soundex code of a word. Then it takes the remaining English word and produces a four-digit representation (shorter codes are extended with zeros), which is a primitive way to preserve the salient features of the phonetic pronunciation of the word. A major problem with Soundex is that it keeps the first letter, thus any error at the beginning of a name will result in a different Soundex code.

3.2.2. Phonex and Phonix. *Phonex* [Lait and Randell 1993] tries to improve the encoding quality by pre-processing names according to their English pronunciation before the encoding. All trailing 's' are removed and various rules are applied to the leading part of a name (for example 'kn' is replaced with 'n', and 'wr' with 'r'). Like Soundex, the leading letter of the transformed name string is kept and the remainder is encoded with numbers (1 letter, 3 digits). On the other hand, *Phonix* algorithm is an improvement for the Phonex and applies more than one hundred transformation rules on groups of letters [Gadd 1990]. Some of these rules are limited to the beginning of a name, some to the end, others to the middle and some will be applied anywhere. Such as, performs the phonetic transformations by replacing certain letter groups with others, replaces the initial letter with v if it is a vowel or the consonant y, and removes the ending sound from the name (the part after the last vowel or y).

3.2.3. Metaphone Algorithm. The *Metaphone algorithm* is also a system for transforming words into codes based on phonetic properties [Philips 1990]. Unlike Soundex, which operates on a letter-by letter scheme, metaphone analyzes both single consonants and groups of letters called diphthongs according to a set of rules for grouping consonants and then maps groups to metaphone codes.

The experiment results from the available misspelling correction techniques demonstrated that each suggestion strategy group has their own strengths [Varol and Bayrak 2009]. For instance, some of the misspelled names were corrected by pattern matching techniques while no phonetic technique was able to do so. The opposite is also true. We therefore employed a methodology to include the strengths of both strategy group techniques while not ignoring the retrieval efficiency. The details of the algorithm are presented in the next section.

4. PERSONAL NAME RECOGNIZING STRATEGY

Personal Name Recognizing Strategy (PNRS) is based on the results of number of strategies that are combined together in order to provide the closest match (Figure 1).

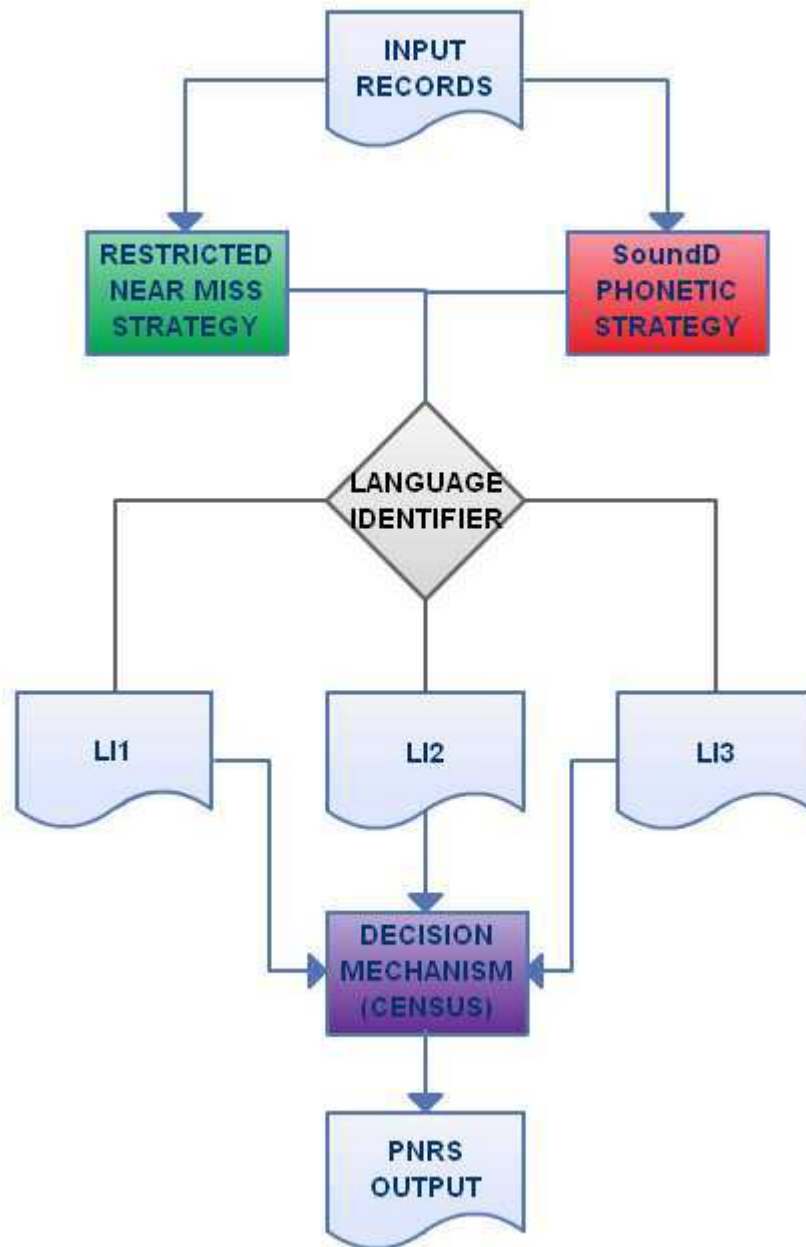


Fig. 1. PNRS Strategy

4.1 String Matching Technique – Restricted Near Miss Strategy

The *Restricted Near Miss Strategy (RNMS)* is a fairly simple way to generate suggestions. Two records are considered near ($t=1$), if they can be made identical by inserting a blank space, interchanging two adjacent letters, changing one letter, deleting one letter, or adding one letter. Also if swapping two distinct letters yield a match, it is also considered within the *RNMS*, where $t=1.5$. At the end, if a valid word is generated using these techniques, then it is added to the temporary suggestion list. Although the threshold value t was selected as one, it changes to one and a half, subsequently to two if there is no initial match with the previous t . In case the first and last characters of a word do not match, we modified our approach to include an extra edit distance.

The main idea behind this is that people generally can get the first character or last character correct when trying to spell a word. These modifications will decrease the total processing time. However, the *RNMS* does not provide the best list of suggestions when a word is truly misspelled. That is where the phonetic strategy takes place.

4.2 Phonetic Matching Technique – SoundD Phonetic Strategy

A phonetic code is a rough approximation of how the word sounds [Cohen et al. 2003]. The English written language is a truly phonetic code, which means each sound in a word is represented by a symbol or sound picture. Since Soundex has less number of phonetic transformation rules (rules are particularly designed for English language) compared to the other algorithms, it provides more logical match for international scope of names among other phonetic strategies [Varol and Bayrak 2009]. Therefore, we have employed a variation of Soundex algorithm as one of our suggestion algorithms. As discussed earlier, a major problem with Soundex is that it keeps the first letter, thus any error at the beginning of a name results in a different Soundex code. This causes to eliminate the valid candidates because of the error in the first letter. Therefore, we have modified the Soundex algorithm with initial letter parsing rules (Table I). Then, the algorithm converts all the letters into numbers according to Table I. Except for the first letter, all zeros (vowels and ‘h’, ‘w’ and ‘y’) are removed and sequences of the same number are reduced to one only (e.g. ‘222’ is replaced with ‘2’). The final code is a 4 digit number (longer codes are cut-off, and shorter codes are extended with zeros). As an example, the Soundex code for ‘martha’ is ‘5630’.

Table I. SoundD Transformation Rules

	Letters	Code
Initial	kn-, gn- pn, ac- or wr-	drop first letter
Initial	x-	change to "s"
Initial	wh-	change to "w"
	D	2 (if in -dge- or -dgi- (Rodgers, Hodgins))
	G	0 (if in -gh- (Houghton))
	<i>Otherwise</i>	
	a e h i o u w y	0
	b f p v	1
	c g j k q s x z	2
	d t	3
	L	4
	m n	5
	R	6

The applied phonetic strategy compares the phonetic code of the misspelled word to all the words in the word list. If the phonetic codes match or within one edit distance of the original word, then the word is added to the temporary suggestion list.

4.3 Language Identifier

Since the input data may contain international scope of names, it is difficult to standardize the phonetic equivalents of certain letters. Therefore, the input data is parsed with a Language Identifier to reflect whether the data contains English based names. Some letters or letter combinations in a name can allow the language to be determined [Fung and Schultz 2008]. For example:

- “tsch”, final “mann” and “witz” are specifically German
- “güi” and “tx” are necessarily Spanish
- “ü” can be only French

More often, several languages can be responsible for a letter or a letter combination. For example, “è” can be French, Spanish or Italian; “th” or final “ck” can be either German or English. Sometimes it can be easier to name the language or the languages in which the letters in question can never occur. For example the string “kie” can be neither French, nor Spanish [Beider and Morse 2008].

Although these rules can be created for all possible languages, since we restrict this project to investigate the personal names in the U.S.A., the current version of the language identification includes about 180 rules for determining whether the name is English, or in another language. The processing of these rules yields one or several languages that could, in principle, be responsible for the spelling entered by the user. Based on the language identifier's results:

- LI1: If the input data contains English based names, the algorithm that is used moves the results of the *Restricted Near Miss Strategy* and *SoundD Phonetic Strategy* into the permanent suggestion pool at the same time and make them of equal weight as long as the threshold t is equal for both algorithms as shown in Algorithm 1.
- LI2: If the input file contains international names, and if there is at least one candidate for a misspelled name within two edit distance score away with the *RNMS*, then the phonetic results are omitted and the permanent suggestion pool consist only the results from the *RNMS* (Algorithm 1). However, pronunciations of other language names can be similar to English as well, such as the name of “Juarez”. Therefore, even for international names, if no candidate is provided by the *RNMS*, then the results of the *SoundD Phonetic Strategy* is moved to the pool.
- LI3: If a suggestion in the pool has the same exact phonetic encoding with the misspelled one and it is the only suggestion that is one edit distance away from the misspelled word with the same phonetic encoding, or there is only one candidate name which is only one edit distance away with both *Restricted Near Miss Strategy* and *SoundD Phonetic Strategy*, the discussed candidate is automatically selected as the possible solution (Algorithm 1).

Algorithm 1. Weighted Hybrid Designation

```

1:  IF Data(d)=English;
2:    Weights =  $\hat{w}$ ;
3:    Read the threshold  $t_d$  from NearMissStrategy
4:    Suggestion list of NearMissStrategy = nms;
5:    Read the threshold  $t_p$  from PhoneticStrategy
6:    Suggestion list of PhoneticStrategy = ps;
7:    LOOP: For each (nmsi = psj)
8:      LOOP: For each ( $t_{di}=1$  &  $t_{pj}=0$ )
9:        New Fragmented Suggestion list of RestrictedNearMissStrategy = nonelist;
10:       New Fragmented Suggestion list of SoundDPhoneticStrategy = pzerolist;
11:       IF: nonelist || pzerolist = 1
12:         Result = Intersect(nonelist, pzerolist); // Final Result
13:       ELSE RankingRate =  $\varpi_i + 2\varpi_j$ ;
14:       END IF
15:     END LOOP
16:     LOOP: For each ( $t_{di}, t_{pj}=1$ )
17:       New Fragmented Suggestion list of RestrictedNearMissStrategy = nonelist;
18:       New Fragmented Suggestion list of SoundDPhoneticStrategy = pzerolist;
19:       IF: nonelist || pzerolist = 1
20:         Result = Intersect(nonelist, pzerolist); // Final Result
21:       ELSE RankingRate =  $\varpi_i + \varpi_j$ ;
22:       END IF
23:     END LOOP
24:     LOOP: For each ( $t_{di}=1.5$  &  $t_{pj}=1$ )
25:       RankingRate =  $2\varpi_i/3 + \varpi_j$ ;
26:     END LOOP
27:     LOOP: For each ( $t_{di}=2$  &  $t_{pj}=1$ )
28:       RankingRate =  $\varpi_i/2 + \varpi_j$ ;
29:     END LOOP
30:   END LOOP
31: END IF
32: IF Data(d)≠English;
33:   LOOP: For each ( $t_{di}=1$ )
34:     RankingRate =  $\varpi_i$ ;
35:   END LOOP
36:   LOOP: For each ( $t_{di}=1.5$ )

```



```

37:           RankingRate =  $2 \varpi_i / 3$ ;
38:   END LOOP
39:   LOOP: For each ( $t_{di}=2$ )
40:           RankingRate =  $\varpi_i / 2$ ;
41:   END LOOP
42: END IF

```

4.4 Decision Mechanism

At the final stage it is possible to see several possible candidate names which have the same weights (ϖ). Relying on only to the edit distances doesn't often provide the desired result. Therefore, we designed our decision mechanism based on the content of the input information and added the U.S Census Bureau decision mechanism to it. The decision mechanism data is a compiled list of popular first and last names which are scored based on the frequency of those names within the United States [Census]. This allows the tool to choose a “best fit” suggestion to eliminate the need for user interaction.

The census score portion of the algorithm is implemented using the following steps:

Step 1: Compare the current suggested names to the census file. If there is a match, store the census score associated with the suggestion.

Step 2: Choose the name with the highest *weighted hybrid designation* score as the “best fit” suggestion.

$$FinalSuggestion = a_0 \varpi_i + a_1 \varpi_j + \varpi_{Census}$$

where, ϖ_{Census} is the normalized (between [0,1]) Census frequency of usage score of the candidate name out of all suggestion list, a_0 and a_1 reflect the coefficients of the *Restricted Near Miss* and *SoundD* phonetic algorithms' weight.

Step 3: If all suggestions are unmatched on the census file, choose the suggestion with the highest weight from the *weighted hybrid designation* algorithm.

5. EXPERIMENTS AND DISCUSSION

In this section, the data set that is tested by the PNRS, pattern matching and phonetic strategies is discussed. The study will be based on the two common measures: *effectiveness* --overall correction rate, *recall* and *precision* --are often referred as retrieval effectiveness.

5.1 Correction Rates of PNRS

The experimental data was a real-life sample which involves personal and associated company names, addresses including zip code, city, state, and phone numbers of individuals. The data set was collected from different sources with variety of entry methods. Out of 173,842 records, dirty data was present in total of 6,128 personal names, which includes misspelled names and some non-ASCII characters. 4,426 records were correctly fixed by PNRS, 163 records were identified as valid names, while 1,539 records were corrected but produced different names as reflected in Figure 2 and Table II.

Out of 6,128 records, 982 of them were not recognized as English names. The *RNMS* failed to provide a suggestion for 19 of the international names. However, with keeping the *SoundD Phonetic Algorithm* in the design, the PNRS was successfully corrected 12 of the names. This means that although the names originated from another language, still common phonetic structures can help in solving matching problems.

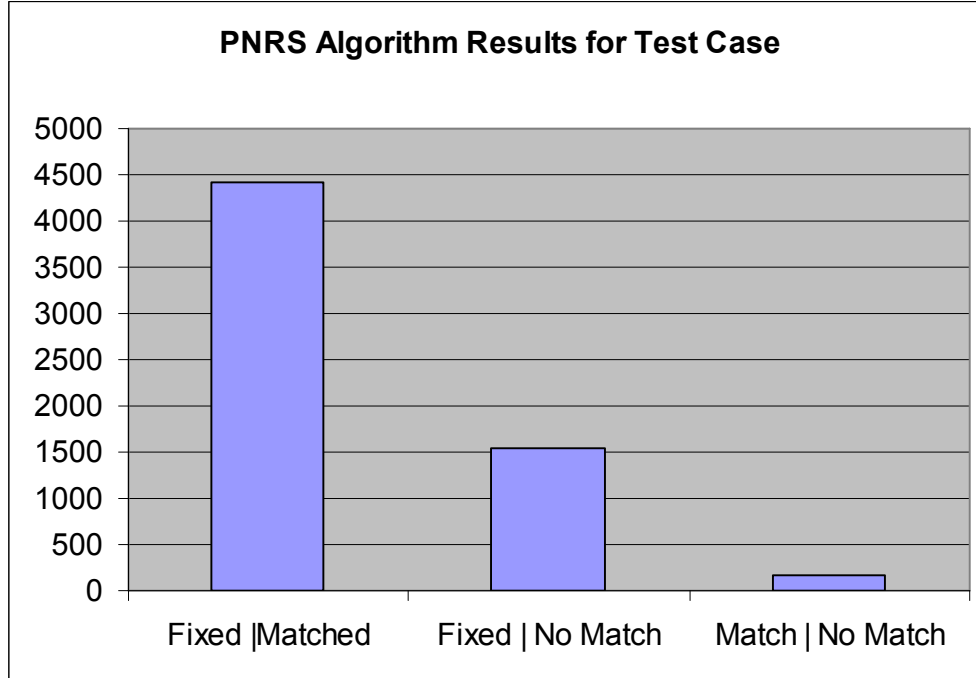


Fig. 2. PNRS Correction Rate for Test Case

Table II. Definition of PNRS Result

Misspelled Name	Original Name	PNRS Suggestion	State of Result
Siteffaannny	Stephanie	Siteffaannny	Match No Match
Luiz	Luis	Luiz	Match No Match
Rca	Rice	Ryan	Fixed No Match
Jaso	Jason	Jason	Fixed Matched

- Fixed | Matched → Exact correction of the misspelled name
- Fixed | No Match → Corrections that provide no match with the original name
- Match | No Match → Either the input is accepted as a valid name or the system failed to provide any suggestions

5.2 Correction Rate Comparison with Other Suggestion Algorithms

In order to evaluate the effectiveness of the tool, experiments were conducted not only on current correction algorithm PNRS, but also on the well known phonetic matching techniques, such as Soundex, Phonex, Phonix, DMetaphone, and the techniques used in dictionary based search (Damerau Edit Distance, LCS, Jaro-Winkler, and n-grams). PNRS averaged 72.4 percent correction rate on the test. Since there is no concrete decision mechanism if there is more than one same score with the Soundex, Phonex, Phonix, DMetaphone, Damerau Edit Distance, LCS, Jaro-Winkler, and n-grams algorithms, it is a challenge to claim which algorithm performs well. However, if we look at the minimum and maximum likelihood of full correction rates (Table III) among all these algorithms and PNRS, we would arguably claim the correction rate of PNRS is satisfactory.

Table III. Minimum and Maximum Correction Rates of the Algorithms

Strategy	Min. and Max. Correction Percentage for Test Case	Strategy	Min. and Max. Correction Percentage for Test Case
Soundex	50.9%-55.6%	Jaro	60.7%-62.6%
Phonex	50.7%-53.5%	2-grams	60.1%-61.1%
Phonix	51.8%-52.1%	3-grams	55.2%-59.3%
DMetaphone	53.7%-54.2%	Restricted NMS	58.6%-65.8%
Damerau Edit-Distance	56.3%-63.1%	SoundD Phonetic Strategy	56.3%-59.2%
LCS	59.7%-63.2%	PNRS	72.4%

5.3 Precision and Recall

In order to reduce overhead, information has to be better organized and should produce relevant results. These results reflect the performance of the each algorithm. Two major metrics commonly associated with Information Retrieval Systems are Precision and Recall [Davis and Goadrich 2006]. Precision can be defined as the number of relevant documents retrieved by a search divided by the total number of documents retrieved by that search. Since algorithms may provide more than one suggestion for a misspelled name, the term relevant documents broadened to include when the correct candidate is one of the suggestions provided by the technique.

As in this case, we have defined Precision as the percentage of correctly detected names in all names suggested by the Suggestion Algorithms.

$$P = \frac{|FM|}{|TS|}$$

Precision measures one aspect of information retrieval overhead for a user performing a particular search. If a search has 90% precision, then 10% of the user effort is the overhead reviewing non-relevant items.

Recall is different than precision. Recall can be defined as the number of relevant documents retrieved by a search divided by the total number of existing relevant documents, or in other words Recall is the percentage of spelling correction rate. Recall gauges how well a system processing a particular query is able to retrieve the relevant items that the user is interested in seeing.

$$R = \frac{|FM|}{|FM| + |FNM| + |MNM|}$$

We have conducted the Precision and Recall to the data set discussed above. We selected the Recall within the interval of [0, 1] and for each 0.1 unit Recall value (11 Recall levels) we plotted the Precision values for the available techniques in the literature. The result is shown in Figure 3.

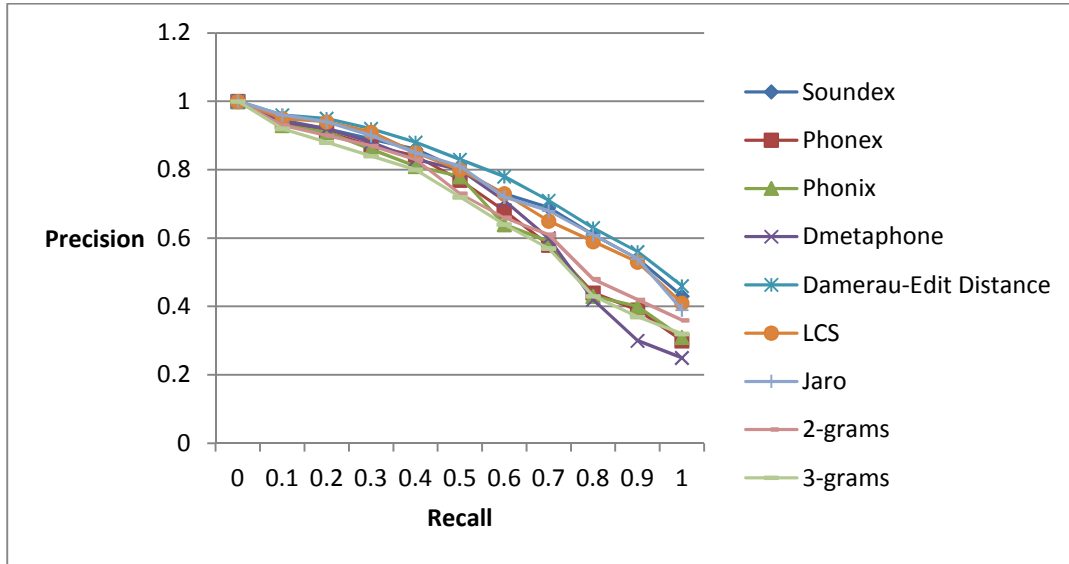


Fig. 3. Precision and Recall for the available algorithms

Ideally, we would like to have 100 percent precision and 100 percent recall. There is, however, often a trade-off between recall and precision. In order to get more relevant items, usually more irrelevant items are retrieved as well. As shown in Figure 4a and 4b, among the algorithms in the literature, both *Damerau Edit Distance* and *Soundex* algorithm have the highest precision values at

all standard recall levels, among their matching strategy groups respectively. The main reason why *Damerau Edit Distance* algorithm has the highest precision values is that the most of the mistakes present in the data set were produced by a single edit (deletion, insertion and replacement) which *Damerau Edit Distance* algorithm is particularly designed to fix. Moreover, because of the variety of names in the US, the more complex phonetic algorithms didn't perform better than the one of the oldest algorithm, *Soundex*.

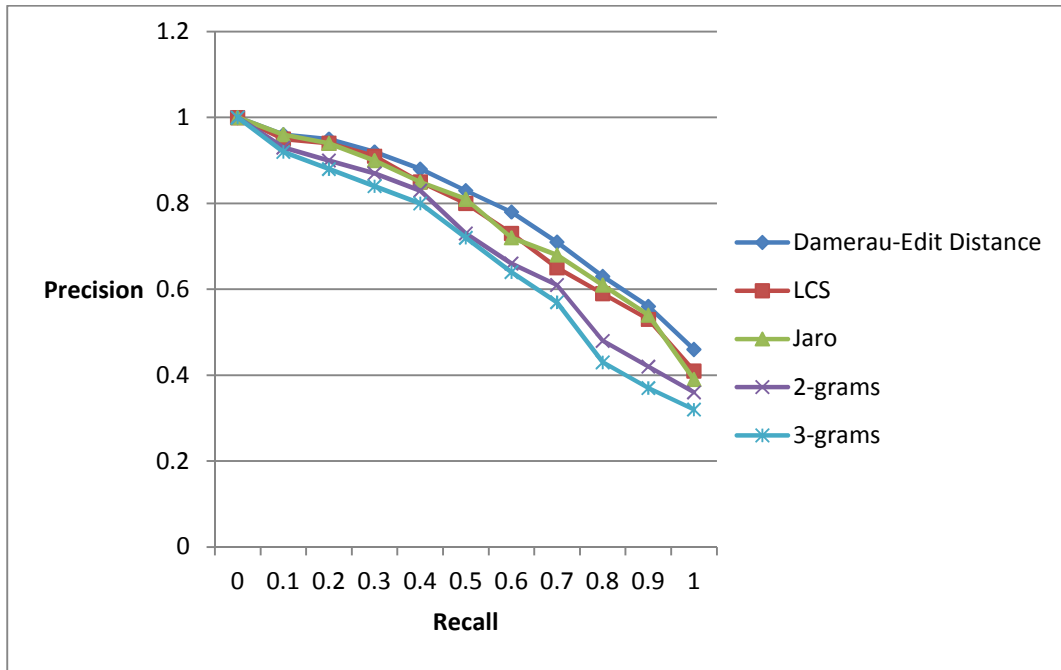


Fig. 4a. Precision and Recall for Pattern Matching Techniques

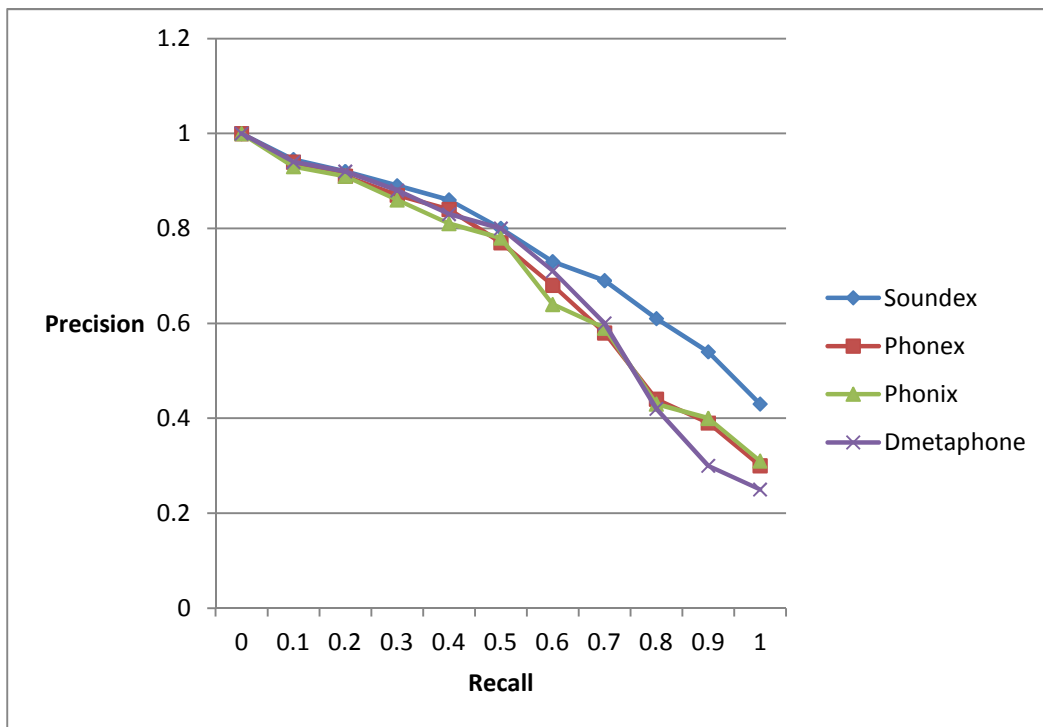


Fig. 4b. Precision and Recall for Phonetic Matching Techniques

For the proposed two different suggestion algorithms, precision values are also calculated to measure system performance and accuracy for comparison. As shown in Figure 5, the *Restricted Near Miss Strategy* produced higher precision values compared to *Damerau-Edit Distance*, because of consideration of adding 1.5 distances cost for swapping two distinct letters and adding another distance cost when the first and last characters of a word do not match. On the other hand, *SoundD Phonetic Strategy* achieved similar results compared to *Soundex* algorithm for the discussed test case.

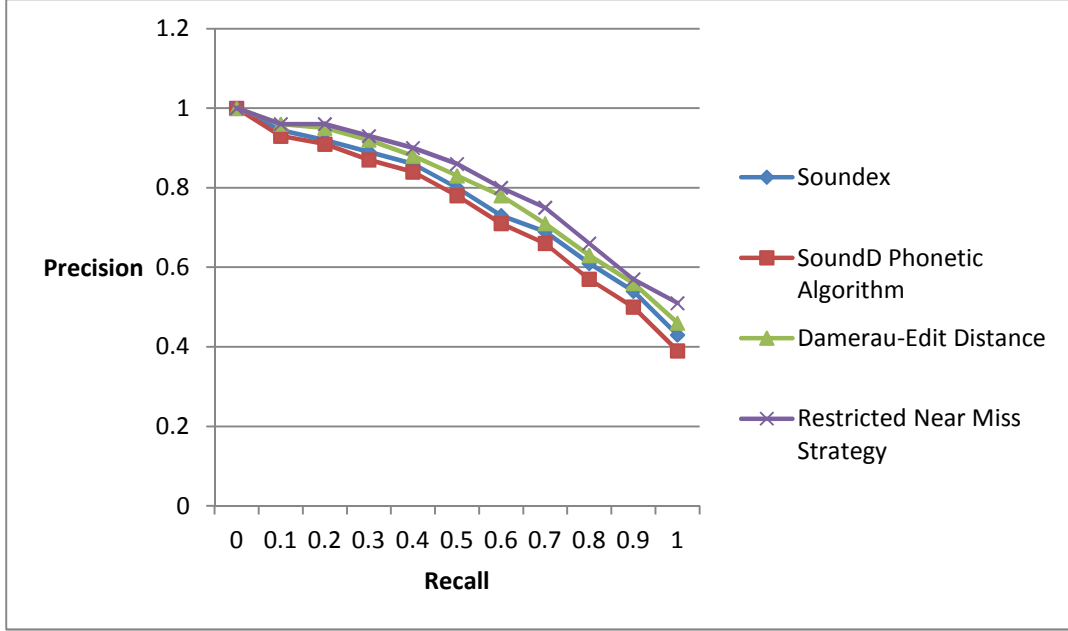


Fig. 5. Precision and Recall for the proposed algorithm

5.4 Fine Tuning Restricted Near Miss Strategy (FT-RNMS)

The result achieved with *Restricted Near Miss Strategy (RNMS)*, as reported in the previous subsections, is among the best for the data set. Encouraged by these observations we carried out additional experiments in order to optimize *RNMS*'s accuracy performance.

As we have discussed, *RNMS* technique depends on numerous parameters including insertion of a blank space (*GapCost*), interchanging two adjacent letters (*SwapCost*), interchanging two distinct letters (*DistSwapCost*), changing one letter (*ChangeCost*), deleting one letter (*DelCost*), or adding one letter (*AddCost*) (default values are 1.5 for *DistSwapCost* and 1 for the other parameters). We applied random search through this 5-dimensional parameter space, repeating the experiment 3,000 times in the data set. Checking only the tiny fraction of possible parameter settings, resulted in a prominent accuracy improvement for the data set when compared to the default settings. The top accuracy results achieved with *GapCost*=0.452, *SwapCost*=0.218, *DistSwapCost*= 0.346, *ChangeCost*=0.254, *DelCost*=0.218, and *AddCost*= 0.258. As we have seen from the results, most of the errors were fixed by swapping the two adjacent letters or deleting an irrelevant character from the misspelled name, followed by changing a character and adding a character. Since the data set is collected from number of sources with different types of entry techniques, we can argue that most of the common misspelling errors in the data entry business are accidentally swapped two adjacent letters and presence of an extra character. The overall correction rates and Precision and Recall values are represented in Table IV and Figure 6.

Table IV. Minimum and Maximum Correction Rates Comparison with FT-RNMS

Strategy	Minimum and Maximum Correction Percentage for Test Case 2
Restricted NMS	58.6%-65.8%
FT-RNMS	62.3%-71.1%
PNRS	72.4%
PNRS (with FT-RNMS)	75.1%

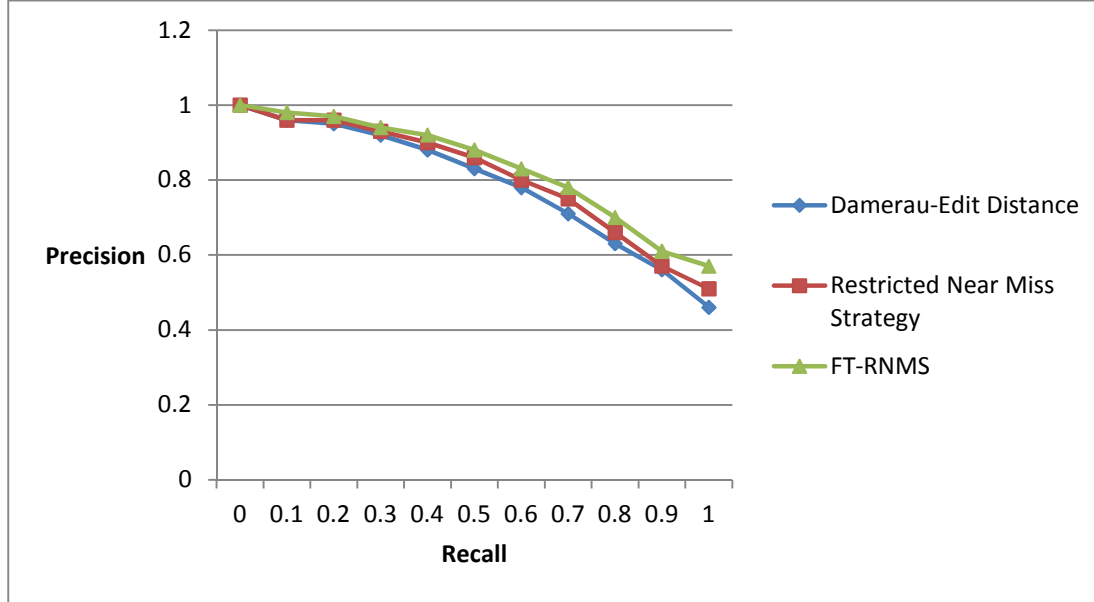


Fig. 6. Precision and Recall for the data set with FT-RNMS

6. CONCLUSIONS

So far we have discussed the characteristics of personal names and the potential sources of errors in them. We have presented an overview of both pattern matching and phonetic based string matching techniques. Experimental results reflected that although the pattern matching techniques provided better correction rates than the phonetic strategies, still phonetic algorithms corrected some mistyped names where pattern matching techniques failed to do so. Therefore, we have created a hybrid matching technique to provide a suggestion for the misspelled name. The matching algorithm (PNRS) combines *Restricted Near Miss Strategy*, *SoundD Phonetic Strategy* and *Census Score* to provide a suggestion. PNRS overcame some of the deficiencies of the other algorithms have when providing a solution for the ill-defined data. First, the algorithm introduced in *RNMS* provided less number of irrelevant results (between 13% and 21% less) compared to other approximate string matching techniques used in dictionary-based search. Second, current techniques heavily rely on only dictionary based search where they employ one of the pattern matching algorithm to provide a suggestion. However, combining the two different matching techniques improved the overall correction rate, since phonetic strategies address phonetic type of errors more accurate than string matching algorithms.

In this study, since the phonetic strategy of this algorithm is particularly designed for English based personal names, other language phonetic structures will be addressed in the near future. Also another future goal is to apply the techniques not only to personal names, but also to addresses and other personal information as well.

ELECTRONIC APPENDIX

The electronic appendix for this article can be accessed in the ACM Digital Library.

REFERENCES

- BECCHETTI, C., AND RICOTTI, L.P. 1999. *Speech Recognition: Theory and C++ Implementation*. John Wiley & Sons.
- BEIDER, A., AND MORSE, S., 2008. Beider-Morse Phonetic Matching: An Alternative to Soundex with Fewer False Hits. *Avotaynu: the International Review of Jewish Genealogy*
- BRILL E., AND MOORE R.C. 2002. An improved error model for noisy channel spelling correction. In: *Proceedings of ACL-2000, the 38th Annual Meeting of the Association for Computational Linguistics*, pp 286-293
- CENSUS Bureau Home Page. 1990. www.census.gov
- CHRISTEN, P. 2006. A Comparison of Personal Name Matching: Techniques and Practical Issues. *ICDM Workshops 2006*: 290-294
- CHRISTEN, P., CHURCHES, T., AND HEGLAND, M. 2004. Febrl - a parallel open source data linkage system. In *PAKDD, Springer LNAI 3056*, 638-647, Sydney 2004.
- CHRISTEN, P., AND GOISER, K. 2006. Quality and complexity measures for data linkage and deduplication. In F. Guillet and H. Hamilton, editors, *Quality Measures in Data Mining, Studies in Computational Intelligence*. Springer
- COHEN, W.W., RAVIKUMAR, P., AND, STEPHEN, E.F. 2003 A comparison of string distance metrics for name-matching tasks. In *Proceedings of IJCAI-03 workshop on information integration on the Web*, pages 73–78, Acapulco.
- DAMERAU, F. J. 1990. Evaluating computer generated domain-oriented vocabularies. *Information Proces. Management*. 26 : 791 – 801
- DAVIS, J., AND GOADRICH, M. 2006. The relationship between Precision-Recall and ROC curves *ICML 2006*: 233-240
- DRAVIS, F. 2002. Information Quality: The Quest for Justification. *Business Intelligence Journal* 7 (2): 44-49.
- DURHAIW, I., LAMB, D.A., AND SAX, J.B. 1983. Spelling correction in user interfaces. *ACM* 26: 764–773
- ECKERSON, W. 2002. *Data Warehousing Special Report: Data quality and the bottom line*. Special report, The Data Warehousing Institute, 101 Communication LLC.
- FRIEDMAN, C., AND SIDELI, R. 1992. Tolerating spelling errors during patient validation. *Computers and Biomedical Research*, 25:486–509.
- FUNG, P., AND SCHULTZ, T. 2008. Multilingual spoken language processing. *IEEE Signal Processing Magazine*, vol. 25, issue 3, pp. 89-97.
- GADD, T. 1990. PHONIX: The algorithm. *Program: automated library and information systems*, 24(4):363–366.
- GONG, R., AND CHAN, T.K. 2006. Syllable alignment: A novel model for phonetic string search. *IEICE Transactions on Information and Systems*, E89-D(1), 332-339.
- HALL, A., AND DOWLING, G.R. 1980. Approximate string matching. *ACM Computing Surveys*, 12(4):381–402.
- JOKINEN, P., TARHIO, J., AND UKKONEN, A. 1996. A comparison of approximate string matching algorithms. *Software – Practice and Experience*, 26(12):1439–1458
- JURZIK, H. 2006. The Ispell and Aspell command line spellcheckers, *WWW.LINUX-MAGAZINE*, 85. issue, 63-66.
- KAHN, B.K., STRONG, D., AND WANG, R. 2002. Information Quality Benchmarks: Product and Service Performance. *Communications of the ACM* 45 (4): 184-192.
- KUKICH, K. 1992. Techniques for Automatically Correcting Words in Text, *ACM Computing Surveys*, vol. 24, No. 4
- LAIT, A., AND RANDELL, B. 1993. An assessment of name matching algorithms. *Technical report*, Department of Computer Science, University of Newcastle upon Tyne, 1993.
- LEE, L., 1999 Measures of distributional similarity. In: *Proceedings of the 37th Annual Meeting of the ACL*
- LEVENSHTAIN, VI. 1965. Binary codes capable of correcting deletions, insertions and reversals. *Doklady Akademii Nauk SSSR* 163 : 845-848, also {1966} Soviet Physics Doklady 10 : 707-710.
- MADNICK, S., WANG, R., LEE, Y., AND ZHU H. 2009. Overview and Framework for data and Information Quality Research. *ACM J. Data Quality* 1 (1): 2. 1-2. 22.
- MARTIN, J., AND RUARK, E.A. 2010. Thy Fiscal Burden of Illegal Immigration on United States Taxpayers. Federation of American Immigration Reform, Fair Horizon Press, July 2010. http://www.fairus.org/site/DocServer/USCostStudy_2010.pdf?docID=4921
- NAVARRO, G. 2001. A guided tour to approximate string matching *ACM Computing Surveys*, 33(1):31–88.
- PHILIPS, L. 1990. Hanging on the metaphone. *Computer Language*, 7(12) : 39-43
- PFREIFER, U., POERSCH, T., AND FUHR, N. 1996. Retrieval effectiveness of proper name search methods. *Information Processing and Management*, 32(6), 667-679.
- RAGHAYAN, V.V., JUNG, G.S., AND BOLLMAN, P. 1989. A critical investigation of recall and precision as measures of retrieval system performance. *ACM Transactions on Information Systems*, 7(3):205–229, July 1989.
- ROGERS, W.P. 1986. Report of the Presidential Commission on the Space Shuttle Challenger Accident. Report, *US Government Accounting Office*, Washington, D.C.
- SOGETI. 2009. www.ie.sogeti.com/en/News--Events/Press-Release/Press-release/
- STRONG, D., LEE, Y., AND WANG, R. 1997. Data quality in context. *Communications of the ACM* 40 (5): 103-110.
- TAGHYA, K., AND STOFISKY, E. 2001 OCRSpell: an interactive spelling correction system for OCR errors in text. *IJDAR*, 3: 125-137
- TRENKLE, J.M. AND VOGT, R.C. 1994. Disambiguation and spelling correction for a neural network based character recognition system. In: *Proceedings of SPIE*. Volume 2181, pp 322-333
- UDUPA, R. AND KUMAR, S. 2010. Hashing-Based Approaches to Spelling Correction of Personal Names. In *Proceedings of The 2010 Conference on Empirical Methods on Natural Language Processing*. October 9-11, 2010, Massachusetts, USA
- ULLMAN, J.R. 1977. A Binary n-Gram Technique for Automatic Correction of Substitution, Deletion, Insertion, and Reversal Errors in Words. *Computer J.*, 20 (2) : 141-147
- VAROL, C., AND BAYRAK C. 2009. Personal Name-based Pattern and Phonetic Matching Techniques: A Survey. *ALAR Conference on Applied Research in Information Technology*, February 13, 2009, Conway, Arkansas, USA

- VAROL, C., AND BAYRAK, C. 2005. Applied Software Engineering Education, *ITHET 2005*, July 6-9, 2005, Santa Domingo, Dominican Republic.
- VAROL, C., BAYRAK, C., AND LUDWIG, R. 2005. Application of Software Engineering Fundamentals: A Hands of Experience, *The 2005 International Conference on Software Engineering Research and Practice*, June 27-30, 2005, Las Vegas, Nevada, USA.
- WANG, R., KON, H., AND MADNICK, S. 1993. Data Quality Requirements Analysis and Modeling. In *Proceedings of the 9th International Conference on Data Engineering*: 670-677.
- WINKLER, W.E. 2006. Overview of record linkage and current research directions. *Technical Report RR2006/02*, US Bureau of the Census.
- YANCEY, W.E. 2005. Evaluating string comparator performance for record linkage. *Technical Report RR2005/05*, US Bureau of the Census.
- YANNAKOUDAKIS, E.J., AND FAWTHROP, D. 1983 The rules of spelling errors. *Information Processing Management* 19 (2) : 87-99
- ZOBEL, J., AND DART, P. 1996. Phonetic string matching: Lessons from information retrieval. In *Proceedings of ACM SIGIR*, 166-172, Zurich, Switzerland.

Received January 2010; revised March 2012; accepted June 2012

Online Appendix to: Hybrid Matching Algorithm for Personal Names

CIHAN VAROL, Sam Houston State University
COSKUN BAYRAK, University of Arkansas at Little Rock

A. APPENDIX SECTION HEAD

Personal Names have been used by several companies, such as People Search services, in order to obtain information about people. However, a good amount of search queries conducted on personal names is misspelled likewise a regular web search. The current correction algorithms are not particularly designed for fixing the problems on personal names. Therefore, we created a hybrid matching algorithm which combines pattern, phonetic matching techniques and use Census scores to fix the ill-defined data.

By doing experiment on real-life misspelled names, we achieve 72.4 percent correction rates with the newly designed algorithm. By fine-tuning the pattern matching technique, the correction rate increased to 75.1 percent